

Reviewer Pack — Persistent H_0 Component Count of Sign-Quotient Row Set Caps...

Entry bc750201aaf2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 00:40:27 UTC

Persistent H_0 Component Count of Sign-Quotient Row Set Caps Real Rank

Entry ID: bc750201aaf2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 00:40:27 UTC

1. Conjecture

Field A (mathematical branch): Persistent homology / Vietoris-Rips topology of finite metric spaces (Vietoris 1927; Edelsbrunner-Letscher-Zomorodian 2002; Carlsson-Zomorodian 2005; Carlsson 2009). The 0-th persistent Betti number $\beta_0(\text{VR}(X, \varepsilon))$ of a finite metric space (X, d) at scale ε is the number of connected components of the ε -thresholded distance graph, computable by Kruskal/union-find in $O(|X|^2 \alpha(|X|))$. Rarely deployed in communication complexity (TDA literature is dominated by data-analysis applications; we are aware of <5 papers connecting VR persistence of communication matrices to log-rank / D^{cc}). **Field B** (complexity object): Real rank $\text{rk}_R(M_f)$ of the $2^k \times 2^k \pm 1$ communication matrix $M_f[x, y] = (-1)^{f(x, y)}$ of a Boolean function $f: \{0, 1\}^k \times \{0, 1\}^k \rightarrow \{0, 1\}$ — the canonical Lovász log-rank / Mehlhorn-Schmidt target for deterministic two-party communication complexity $D^{\text{cc}}(f) \geq \log_2 \text{rk}_R(M_f)$.

Statement:

Let $X_f^{\pm} \subset \{\pm 1\}^{2k} / \{\pm 1\}$ be the set of distinct rows of M_f modulo global sign, equipped with the projective Hamming distance $d_{\pm}(r, s) = \min(d_H(r, s), 2^k - d_H(r, s))$. Let $\tau_{\pm}(M_f) := \beta_0(\text{VR}(X_f^{\pm}, 2^{\{k-2\}}))$, the number of components of the graph linking r, s whenever $d_{\pm}(r, s) \leq 2^{\{k-2\}}$. Then for every Boolean f and every $k \in \{3, 4, 5, 6\}$, $\tau_{\pm}(M_f) \leq \text{rk}_R(M_f)$; a single (k, f) with $\tau_{\pm}(M_f) > \text{rk}_R(M_f)$ refutes.

Rationale (proposer's reasoning):

Rank- $r \pm 1$ matrices have all rows lying in an r -dimensional subspace of $\mathbb{R}^N$, and ± 1 vectors in such a subspace are constrained to a structured (centrally-symmetric) finite set whose pairwise distances avoid the ‘mid-Hamming’ regime around $N/2$ only by sitting in a small affine pattern; persistence at scale $N/4$ should therefore not be able to manufacture more than r distinct components after the $\pm$ -quotient. This bridges the topological ‘shape-counting’ of the row metric space to the algebraic rank-bound side of log-rank, a structural inequality complementary to spectral approaches and untouched by natural-proofs (rk_R is the property being lower-bounded, not a hardness predicate on truth tables) and untouched by algebrization (no oracle relativization is used).

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5b190f5fef43d120

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 4 values of $k \in \{3,4,5,6\} \times 30$ seeds $\times$ 6 function families (~ 720 instances), compute $\tau_{\pm}(M_f)$ and $rk_R(M_f)$. Conjecture is SUPPORTED iff $\tau_{\pm} \leq rk_R$ on $\geq 99\%$ of instances AND zero instances violate by margin >0 ; FALSIFIED iff any single instance yields $\tau_{\pm} > rk_R$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - persistent homology Vietoris-Rips communication matrix log-rank - topological data analysis Boolean function rank communication complexity - persistent Betti number sign rank Hamming distance rows

Top relevant hits considered: - [s2:10.4230/LIPIcs.ESA.2024.29] A Euclidean Embedding for Computing Persistent Homology with Gaussian Kernels - [s2:609241bc94e187c943bd6d4ec6bcf0c1ababa87c] Topology in sensor networks - [s2:2505.16879] How high is ‘high’? Rethinking the roles of dimensionality in topological data analysis and manifold learning - [s2:10.1007/PL00009265] Harmonic Analysis, Real Approximation, and the Communication Complexity of Boolean Functions - [s2:f23504fcde864a63a36bc0be33c3a3f97d615906] The Nature of Migration and Its Impact on Families in Peru

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import sys
import math
import itertools
import collections

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

def generate_random_f(k):
    return [random.choice([-1, 1]) for _ in range(2**k)]

def generate_structured_f(k, family):
    if family == 'IP_k':
        return [sum(f[i] * f[j] for i in range(k) for j in range(i+1, k)) for f in itertools.product([-1, 1], repeat=k)]
    elif family == 'EQ_k':
        return [sum(f[i] * f[j] for i in range(k) for j in range(i+1, k)) - sum(f[k+i] * f[k+j] for i in range(k) for j in range(i+1, k)) for f in itertools.product([-1, 1], repeat=2*k)]
    elif family == 'GT_k':
        return [sum(f[i] * f[j] for i in range(k) for j in range(i+1, k)) - sum(f[k+i] * f[k+j] for i in range(k) for j in range(i+1, k)) for f in itertools.product([-1, 1], repeat=2*k)]
    elif family == 'AND-of-XORs':
        return [sum(f[i] & f[j] for i in range(2**k)) for f in itertools.product([-1, 1], repeat=2**k)]
    elif family == 'MAJ_k(x XOR y)':
        return [sum(f[i] * (f[i] ^ f[j]) for i in range(k) for j in range(i+1, k)) for f in itertools.product([-1, 1], repeat=k)]
    else:
        raise ValueError("Unsupported family")

def generate_low_rank_matrix(n, r):
    generators = [generate_random_f(math.ceil(math.log2(n))) for _ in range(r)]
    matrix = [[sum(g[i] * g[j] for j in range(r)) % 2 for i in range(n)] for g in generators]
    return matrix

def projective_hamming_distance(row1, row2):
    return min(sum(a != b for a, b in zip(row1, row2)), len(row1) - sum(a != b for a, b in zip(row1, row2)))

def union_find(n):
    parent = list(range(n))
    rank = [0] * n

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(x, y):
        rootX = find(x)
        rootY = find(y)
        if rootX != rootY:
            if rank[rootX] > rank[rootY]:
                parent[rootY] = rootX
            elif rank[rootX] < rank[rootY]:
                parent[rootX] = rootY
            else:
                parent[rootY] = rootX
                rank[rootX] += 1
        return union, find

    return union, find

def gaussian_elimination(matrix):
    n = len(matrix)
    m = len(matrix[0])
    augmented_matrix = [row + [Fraction(0)] for row in matrix]

    for i in range(n):
        max_row = i
        for j in range(i+1, n):

```

```

        if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
            max_row = j

    augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

    pivot = augmented_matrix[i][i]
    for j in range(m):
        augmented_matrix[i][j] /= pivot

    for j in range(n):
        if i != j:
            factor = augmented_matrix[j][i]
            for k in range(m):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    rank = 0
    for row in augmented_matrix:
        if any(row[i] != Fraction(0) for i in range(m)):
            rank += 1

    return rank

def compute_rk_R(matrix):
    n = len(matrix)
    m = len(matrix[0])
    integer_matrix = [[Fraction(matrix[i][j]) for j in range(m)] for i in range(n)]
    return gaussian_elimination(integer_matrix)

def compute_tau_plus_minus(M, k):
    n = len(M)
    union, find = union_find(n)

    for i in range(n):
        for j in range(i+1, n):
            d = projective_hamming_distance(M[i], M[j])
            if d <= 2**(k-2):
                union(i, j)

    components = set()
    for i in range(n):
        components.add(find(i))

    return len(components)

k_values = [3, 4, 5, 6]
instances_tested = 0
total_tau_plus_minus = 0
total_rk_R = 0

for k in k_values:
    n = 2**k

    for _ in range(30):
        f = generate_random_f(k)
        M_random = [generate_random_f(n) for _ in range(n)]
        M_structured = generate_structured_f(k, random.choice(['IP_k', 'EQ_k', 'GT_k', 'AND-or

```

```

M_low_rank = generate_low_rank_matrix(n, 2**k)

for M in [M_random, M_structured, M_low_rank]:
    M = [[Fraction(row[i]) for i in range(n)] for row in M]
    unique_rows = list(set(tuple(row) for row in M))
    tau_plus_minus = compute_tau_plus_minus(unique_rows, k)
    rk_R = compute_rk_R(M)

    total_tau_plus_minus += tau_plus_minus
    total_rk_R += rk_R
    instances_tested += 1

mean_tau_plus_minus = total_tau_plus_minus / instances_tested
mean_rk_R = total_rk_R / instances_tested

conjecture_holds = all(tau_plus_minus <= rk_R for tau_plus_minus, rk_R in zip([mean_tau_plus_minus, mean_rk_R]))

return {
    "metric_name": "tau_plus_minus",
    "metric_value": mean_tau_plus_minus,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_tau_plus_minus = sum(r["metric_value"] for r in results) / len(results)
    std_tau_plus_minus = math.sqrt(sum((r["metric_value"] - mean_tau_plus_minus)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_tau_plus_minus} std={std_tau_plus_minus} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSEIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7aad0d36.py", line 185, in <module>

```

result = run_trial(seed)
^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7aad0d36.py", line 163, in run_trial
    M.append([Fraction(row[i]) for i in range(n)])
                ~~~^^^

```

IndexError: tuple index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError before producing any RESULT line, so neither the support nor the falsification condition could be evaluated. Per rule 2, a crashed test yields INCONCLUSIVE. | next: Fix the row-construction bug (tuple index out of range when building M from row entries) and re-run the full 4×30×6 grid to obtain $\tau_{\pm}$ and rk_R counts.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	254978
2	preregistration	claude_max	opus	0	0	5917
3	novelty	claude_max	opus	0	0	3335
4	novelty	claude_max	opus	0	0	8850
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22751
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19991
7	judge	claude_max	opus	0	0	5023

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 320845 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/bc750201aaf2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bc750201aaf2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bc750201aaf2.tar.gz` (if generated)